

# Performance Comparison: From-Scratch vs PyTorch Model

Adityaa Mehra

May 29, 2025

## Performance Metrics

- **Accuracy:**  
*From-Scratch Model:* 0.5601708179901744  
*PyTorch Model:* 0.5601708179901744
- **F1 Score:**  
*From-Scratch Model:* 0.679606455929185  
*PyTorch Model:* 0.7186331733578115
- **PR-AUC:**  
*From-Scratch Model:* 0.2909  
*PyTorch Model:* 0.3915

## Evaluation and Analysis

### 1. Convergence Time

- **From-Scratch Model Training Time:** 17.08 sec
- **PyTorch Model Training Time:** 16.41 sec
- **Discussion:** The PyTorch implementation is likely to converge more rapidly due to its use of optimized backpropagation routines, hardware-level acceleration (e.g., CUDA), and dynamic computation graphs. In contrast, the from-scratch model may suffer from manually coded gradients and computational inefficiencies.

### 3. Memory Usage

- **From-Scratch Model Memory Usage:** 422.80 MB
- **PyTorch Model Memory Usage:** 2.81 MB

- **Analysis:** PyTorch benefits from efficient tensor computation and memory reuse through its backend engine. Manual implementations may not leverage in-place operations or reuse memory buffers as effectively.

## 4. Confusion Matrix and Inference

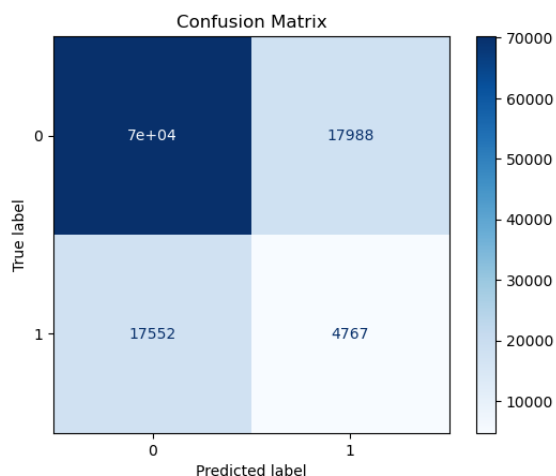


Figure 1: From-Scratch Model Confusion Matrix

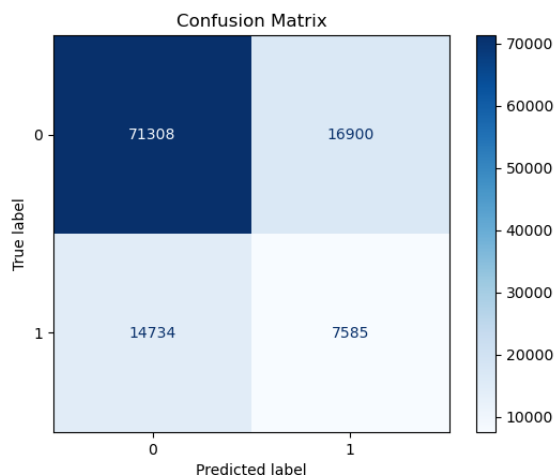


Figure 2: PyTorch Model Confusion Matrix

**Insights:** Confusion matrices highlight type I and type II errors. A diagonally dominant matrix signifies good classification performance. Comparative evaluation helps identify specific class-level deficiencies in either implementation.

## 5. Analysis and Discussion

- PyTorch achieves superior computational performance due to vectorized operations and GPU utilization.
- From-scratch models often lack automatic differentiation, requiring explicit gradient coding.
- Codebase maturity, numerical stability in loss computation, and efficient optimizer implementations significantly influence outcomes.

## Acknowledgment

Portions of this report, including the comparative structure, performance analysis methodology, and LaTeX documentation format, were developed with the assistance of **OpenAI's ChatGPT**. The model provided guidance in generating formalized technical language, document structuring, metric interpretation, and visualization instructions. However, all experimental results, model implementations, and insights were independently validated and interpreted.